Chapter 5 worked solutions

DS05-01 • Traversals and height

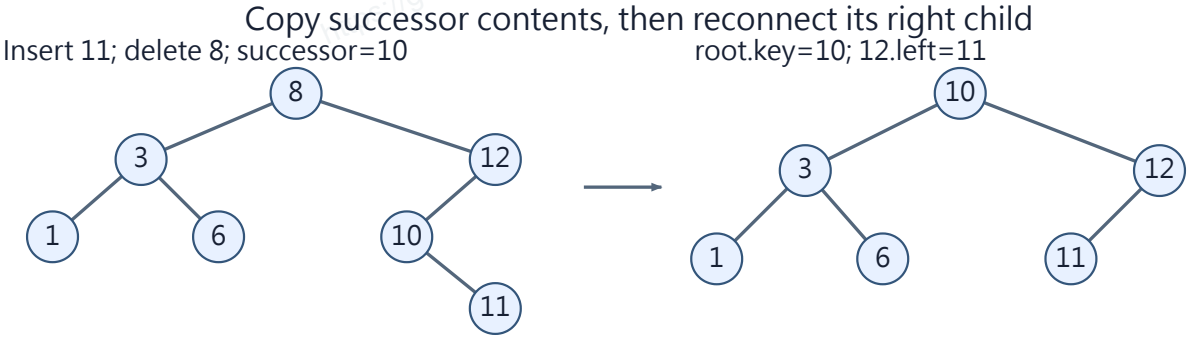
Preorder is `8,3,1,6,12,10`, inorder `1,3,6,8,10,12`, postorder `1,6,3,10,12,8`, and level order `8,3,12,1,6,10`. Root depth is 0. Nodes 3 and 12 have depth 1. Nodes 1,6,10 have depth 2. Height is 2 and maximum width is 3. DFS uses O(h+1) auxiliary stack space for a nonempty tree, constant in this example. BFS uses O(w) queue space. For an unknown shape either can require O(n). This balanced example does not prove a universal logarithmic-space bound.

DS05-02 • BST insertion and two-child deletion

Node 11 becomes 10's right child. The successor of 8 is 10. Copying its contents changes the root key to 10, then removes the original node 10 to the left of 12. Its only child 11 becomes 12's left child. The final tree is:

```
      10
     /  \
    3    12
   / \   /
  1   6 11
```

Inorder is `1,3,6,10,11,12`. A handle to the old root now sees key 10, while a handle to the original node 10 becomes invalid even though key 10 remains. Stable element-identity guarantees require transplanting nodes rather than arbitrary content copying.



DS05-03 • Heap updates

Append 1 at index 6, giving `[2,5,4,9,7,8,1]`. Swap indices 6 and 2 to get `[2,5,1,9,7,8,4]`, then 2 and 0 to get `[1,5,2,9,7,8,4]`. Delete-min moves final value 4 to the root and shrinks to `[4,5,2,9,7,8]`. Swap the root with smaller child 2, giving `[2,5,4,9,7,8]`. Child 8 below index 2 is not smaller, so stop. Each operation follows only a height-length path.

Chapter 5 worked solutions (continued)

DS05-04 • Prove linear heap construction

Multiplying operation count by the maximum individual bound proves O(n log n), without an Ω(n log n) lower bound. Most nodes are near leaves and cannot descend log n levels. There are O(n/2^h) nodes of height h, so total work is `O(nΣh/2^h)=O(n)`. Reading or initializing n elements requires Ω(n), giving Θ(n). With individual insertions of successively smaller keys, every new node rises to the root. Total cost is `Σ_{i=1}^n Θ(log i)=Θ(n log n)`.

DS05-05 • From ordered sources to a balanced tree

A heap of source heads repeatedly emits the minimum and replenishes that source. Aggregate the two 7 records into value 2, producing seven unique keys 2,5,7,11,13,17,19; other values are 1. Count, replay, and construct inorder: the first three form 5(2,7), next key 11 becomes root, and the last three form 17(13,19). Root.parent=null; 5 and 17 have parent 11; 2,7 have parent 5; 13,19 have parent 17. Leaves have height 0, their parents 1, and root 2.

With N=8 total records, merging costs O(k+N log(k+1)), linear for fixed k. Construction consumes each unique record once in Θ(U), with O(log(U+1)) extra stack. Replaying and counting change only constants. A nonrepeatable source of unknown U needs buffering. Halving counts proves logarithmic height, and inorder consumption proves global BST order. Ordinary insertion of sorted keys can instead take $\Theta(U^2)$.

Successor(7) climbs to 5 after completing its right subtree, then to 11, whose left child is 5; the answer is 11. Successor(19) climbs through 17 and 11 beyond the root, returning null. Each call costs O(h+1) worst-case, while a complete ordered traversal uses each edge at most once each way and costs Θ(U).

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study